

Network Intrusion Detection & Prevention Systems Exploitation

Use of Suricata in IDS mode (Part 1)

Lab Objective

The aim of this Lab is to install, configure, and use Suricata, which is a free and open source, network threat detection engine that helps protect networks against threats. It actively monitors network traffic and detects malicious behavior based on written rules.

Suricata is capable of real time intrusion detection (IDS), inline intrusion prevention (IPS), network security monitoring (NSM) and offline pcap processing.

After exploring and understanding how to configure Suricata, you will be asked to implement some security policies to respond to connections on your machine and prevent attacks **[1]**.

I. Running Suricata in IDS mode

The aim of this section is to run Suricata in passive IDS mode.

16. Many rules are enabled by the default installation within the file `/var/lib/suricata/rules/suricata.rules`

Count the number of these detection rules using the command:

```
# wc -l /var/lib/suricata/rules/suricata.rules
```

17. Open the rules file and check the existence of the following detection rule. Refer to the appendix and to the suricata rule webpage to understand each element of this rule:

```
# nano /var/lib/suricata/rules/suricata.rules
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"ET USER_AGENTS Suspicious User Agent  
(BlackSun)"; flow:to_server,established; http.user_agent; content:"BlackSun"; nocase; depth:8;  
reference:url,www.bitdefender.com/VIRUS-1000328-en--Trojan.Pws.Wow.NCY.html;  
reference:url,doc.emergingthreats.net/bin/view/Main/2008983; classtype:trojan-activity; sid:2008983; rev:8;  
metadata:created_at 2010_07_30, former_category HUNTING, updated_at 2020_10_13;)
```

Note that this rule configures Suricata to issue an alert each time that the HTTP user-agent field in the captured packet contains the string "BlackSun". User-agent strings are sometimes used by malware

authors as an authentication token, so that the command-and-control server will not issue commands to computers that make requests unless the correct user-agent string is specified by the client in the HTTP session. This is one way malware authors evade malware detectors. Therefore, these user-agent strings can be very good indicators of malware presence on a system.

Indicate the role of the following modifiers:

- flow:to_server,established → only match traffic where an established TCP session exists, and the traffic is from a TCP client to a TCP server
- http.user_agent → HTTP User-Agent header value
- Content → Instruct the IDS/IPS engine to examine the application layer content (the payload) of a packet for the specific data "BlackSun".
- nocase → This content will match any capitalization of the word "BlackSun".
- depth:8 → Is used to specify where it will cease looking for a content match of the word "BlackSun" within a packet payload. Note that the payload begins at byte 0, rather than byte 1.
- reference → the reference with the detected attack
- classtype → Allows to assign detection rules to categories based upon the type of activity they are attempting to detect (the classification name exist in classification.config file)

- sid → Signature Identifier (sid) which is used to uniquely identify rules. Each rule must have a unique SID, which is simply a numeric value.
- Rev → Is used to denote when a rule has been changed.

18. From the Virtual machine, generate a HTTP traffic to www.google.com website using the user-agent string "BlackSun"

```
# yum install curl
# curl -A "BlackSun" www.google.com
```

Indication: curl is a tool to transfer data from or to a server, using one of the supported protocols (FTP, FTPS, HTTP, HTTPS, IMAP, IMAPS, LDAP, LDAPS, POP3, POP3S, ...)

The -A option specifies the User-Agent string to send to the HTTP server.

19. The attempted connection should generate the following alert in `/var/log/suricata/fast.log` file

```
#tail -f /var/log/suricata/fast.log
```

```
08/24/2021-18:09:11.411343  [**] [1:2008983:8] ET USER_AGENTS Suspicious User Agent (BlackSun) [**] [Classification: A Network Trojan was detected] [Priority: 1] {TCP} 192.168.4.36:57634 -> 142.250.180.100:80
```

II. Testing Suricata response to malicious traffic

Certain cyber-attacks (e.g., synflood attacks, UDP flooding, ...) may use falsified IP addresses or launch attacks using networks which are known by their maliciousness. To counteract these attacks, many security organizations define a list of IP addresses that need to be monitored as Spamhaus. The Spamhaus DROP (Don't Route Or Peer) lists are advisory "drop all traffic" lists, consisting of netblocks (groups of ip addresses) that are "hijacked" or leased by professional spam or cyber-crime operations (used for dissemination of malware, trojan downloaders, botnet controllers). The DROP lists are a tiny subset of the SBL (Spamhaus Block List), designed for use by firewalls and routing equipment to filter out the malicious traffic from these netblocks.

20. Check the existence of the following rule in the file `/var/lib/suricata/rules/suricata.rules` which serves in the detection of suspicious IP addresses

```
alert ip [2.57.234.0/23,2.58.148.0/22,5.42.199.0/24,5.134.128.0/19,5.183.60.0/22,5.188.10.0/23,5.188.236.0/23,
24.137.16.0/20,24.170.208.0/20,24.233.0.0/19,24.236.0.0/19,27.123.208.0/22,27.126.160.0/20,27.146.0.0/16,31.24.81.0/2
4,31.41.244.0/24,31.217.252.0/24,31.222.236.0/24,36.0.8.0/21,36.37.48.0/20] any -> $HOME_NET any (msg:"ET DROP
Spamhaus DROP Listed Traffic Inbound group 1"; reference:url,www.spamhaus.org/drop/drop.lasso; threshold: type limit, track
by_src, seconds 3600, count 1; classtype:misc-attack; flowbits:set,ET.Evil; flowbits:set,ET.DROPIP; sid:2400000; rev:3821;
metadata:affected_product Any, attack_target Any, deployment Perimeter, tag Dshield, signature_severity Minor, created_at
2010_12_30, updated_at 2023_12_04;)
```

21. On your VM, install hping3 tool to perform an SYN Flooding DoS attack.

22. `yum install`

```
https://epel.mirror.liquidtelecom.com/epel/8/Everything/x86_64/Packages/h/hping3-0.0.20051105-33.el8.x86_64.rpm
```

23. Generate an SYN Flooding DOS attack on a chosen target machine (The target should be in the same local network. You can choose your main host, or one of your neighbors' VMs). In the following command, the target is 192.168.4.20 and the chosen port is 2020 (a closed port) to avoid generating a true DoS attack. Hopefully, this this does not prevent attack detection and alert generation by Suricata. The source addresses (chosen randomly in this command) will coincide with addresses in the Spamhaus Block List.

```
# hping3 -S -p 2020 --flood --rand-source 192.168.4.20
```

Refer to `man hping3` to understand the meaning of every option in the executed command

24. While hping3 is running, tail the alert logs on Suricata to display the generated alerts

```
[root@centos8 centos]# tail -f /var/log/suricata/fast.log
08/24/2021-18:56:53.598858  [**] [1:2400012:2980] ET DROP Spamhaus DROP Listed Traffic Inbound group 13 [**]
[Classification: Misc Attack] [Priority: 2] {TCP} 137.114.57.38:58348 -> 192.168.4.20:2020
08/24/2021-18:56:53.616846  [**] [1:2400004:2980] ET DROP Spamhaus DROP Listed Traffic Inbound group 5 [**]
[Classification: Misc Attack] [Priority: 2] {TCP} 86.106.95.221:58987 -> 192.168.4.20:2020
08/24/2021-18:56:53.619186  [**] [1:2400008:2980] ET DROP Spamhaus DROP Listed Traffic Inbound group 9 [**]
[Classification: Misc Attack] [Priority: 2] {TCP} 102.240.217.55:59141 -> 192.168.4.20:2020
08/24/2021-18:56:53.610235  [**] [1:2400008:2980] ET DROP Spamhaus DROP Listed Traffic Inbound group 9 [**]
[Classification: Misc Attack] [Priority: 2] {TCP} 101.192.125.95:58749 -> 192.168.4.20:2020
08/24/2021-18:56:53.625998  [**] [1:2400012:2980] ET DROP Spamhaus DROP Listed Traffic Inbound group 13 [**]
[Classification: Misc Attack] [Priority: 2] {TCP} 137.105.53.161:59462 -> 192.168.4.20:2020
08/24/2021-18:56:53.628675  [**] [1:2400018:2980] ET DROP Spamhaus DROP Listed Traffic Inbound group 19 [**]
[Classification: Misc Attack] [Priority: 2] {TCP} 168.80.214.52:59619 -> 192.168.4.20:2020
```

III. Adding our own rules to Suricata

Detecting Echo Requests

In this step, we will add a rule to suricata to detect the use of icmp traffic. For that, you are asked to follow the next steps:

25. Create a rule file `custom.rules` under the directory `/var/lib/suricata/rules/`

```
# vi /var/lib/suricata/rules/custom.rules
```

and add the following rule:

```
alert icmp $HOME_NET any -> $EXTERNAL_NET any (icode:0; itype:8; msg: "Ping Detected"; sid:1000001; rev:1;)
```

26. Specify `custom.rules` as a rule file in the configuration file `/etc/suricata/suricata.yaml`

```
##  
## Configure Suricata to load Suricata-Update  
##  
  
default-rule-path: /var/lib/suricata/rules  
  
rule-files:  
- suricata.rules  
- custom.rules
```

27. Stop suricata (CTRL + C) and then restart it

```
# suricata -c /etc/suricata/suricata.yaml -v -i enp0s3
```

28. Open new terminal and ping to the external IP address (e.g., 8.8.8.8), check the Suricata log file fast.log, and verify that your rule has been considered by your IDS.

```
# ping 8.8.8.8
```

```
# tail -f /var/log/suricata/fast.log
```

```
[root@centos8 centos]# tail -f /var/log/suricata/fast.log  
08/26/2021-18:33:52.342363  [**] [1:2:1] Ping Detected [**] [Classification: (nu  
ll)] [Priority: 3] {ICMP} 192.168.4.36:8 -> 8.8.8.8:0  
08/26/2021-18:33:53.347383  [**] [1:2:1] Ping Detected [**] [Classification: (nu  
ll)] [Priority: 3] {ICMP} 192.168.4.36:8 -> 8.8.8.8:0  
08/26/2021-18:33:54.348290  [**] [1:2:1] Ping Detected [**] [Classification: (nu  
ll)] [Priority: 3] {ICMP} 192.168.4.36:8 -> 8.8.8.8:0
```

Analyzing PCAP file and detecting phishing attacks

The objective is to analyze a pcap traffic capture, detect a phishing attacks on HHTP and DNS queries, and write custom rules in SURICATA to detect such an attack.

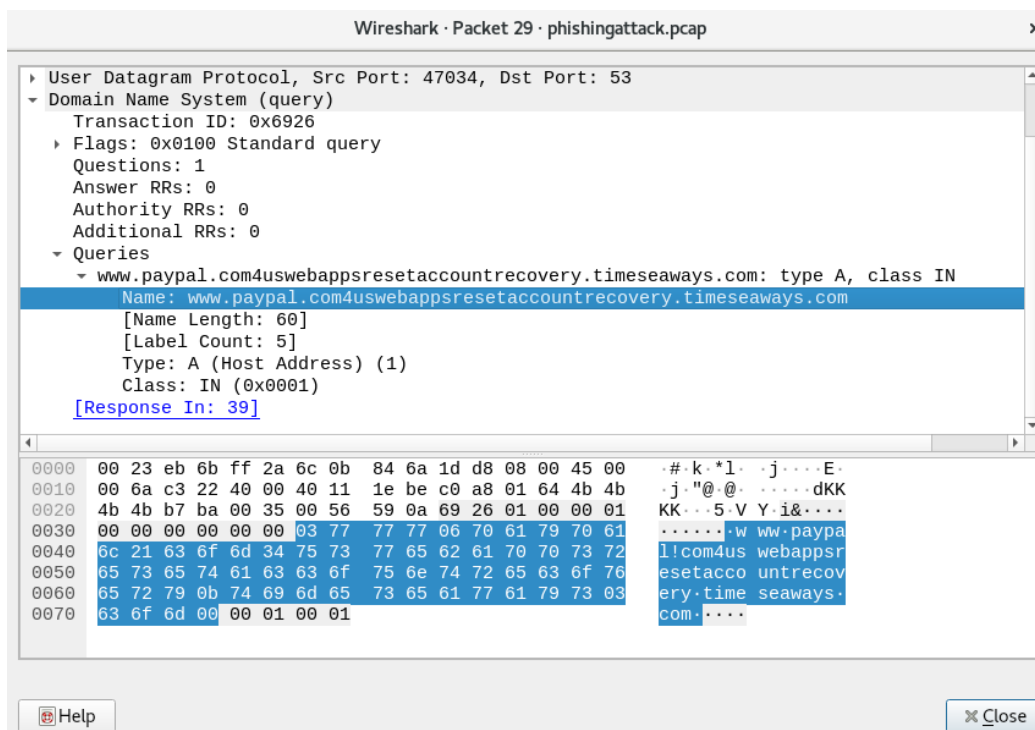
29. Download the pcap file and open it using wireshark

```
# wget https://github.com/jishnusaaurav/Phishing-attack-PCAP-analysis-  
using-scapy/raw/master/phishingattack.pcap
```

30. Open packet 29, which represents a DNS query, and examine the content of the DNS protocol, and observe the following DNS query:

```
www.paypal.com4uswebappsresetaccountrecovery.timeseaways.com
```

Is it a phishing attack? If yes, how it works?



31. To write Suricata signature for this attack, we need to find and handle necessary patterns from the payload. We can write a signature to detect **any value after** the real **PayPal domain** (paypal.com). For that, we use the “**isdataat**” **relative** keyword (See appendix for a better description of the keyword usage).

In the rule file custom.rules under the directory /var/lib/suricata/rules/ add the following signature to detect the phishing attack.

```
alert dns $HOME_NET any -> $EXTERNAL_NET 53 (msg:"Paypal Phishing Detected";
dns_query; content:"paypal.com"; nocase; isdataat:1,relative; sid:1000003; rev:1;)
```

The traffic in phishingattack.pcap contains patterns related to other attacks. In order to only test the above phishing attack signature and avoid generating other alerts when loading this pcap file, you can deactivate the rule file suricata.rules in the configuration file /etc/suricata/suricata.yaml (and only keep custom.rules active).

```
default-rule-path: /var/lib/suricata/ru  
rule-files:  
- suricata.rules  
- custom.rules  
##  
## Auxiliary configuration files.  
##
```

32. Stop suricata (CTRL + C) and then restart it

```
# suricata -c /etc/suricata/suricata.yaml -v -i enp0s3
```

33. Open new terminal and replay the pcap file (phishingattack.pcap) over the network interface enp0s3

using the command:

```
# tcpreplay -i enp0s3 phishingattack.pcap
```

```
tcpreplay-edit -C -i enp0s3 phishingattack.pcap
```

34. Check the Suricata log file fast.log, and verify that the attack was successfully detected:

```
# tail -f /var/log/suricata/fast.log
```

```
[root@centos8 centos]# tail -f /var/log/suricata/fast.log
09/02/2021-17:05:14.899430  [**] [1:1000003:1] Paypal Phishing Detected [**] [Classification: (null)] [Priority: 3] {UDP} 192.168.1.100:47695 -> 75.75.75.75:53
09/02/2021-17:05:14.899430  [**] [1:2200075:2] SURICATA UDPv4 invalid checksum [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {UDP} 192.168.1.100:47695 -> 75.75.75.75:53
09/02/2021-17:05:16.429580  [**] [1:1000003:1] Paypal Phishing Detected [**] [Classification: (null)] [Priority: 3] {UDP} 192.168.1.100:47034 -> 75.75.75.75:53
09/02/2021-17:05:16.429580  [**] [1:2200075:2] SURICATA UDPv4 invalid checksum [**] [Classification: Generic Protocol Command Decode] [Priority: 3] {UDP} 192.168.1.100:47034 -> 75.75.75.75:53
```

Appendix

Suricata Rules Format

The Suricata rules/signature consists of the following parts:

- action: determines what happens when the signature matches (e.g., alert, drop)
- header : protocol + source address + source port -> destination address + destination port (defines the protocol, IP addresses, ports and direction of the rule)
- rule options: meta-settings + payload keywords + protocol keywords (defines the specifics of the rule)

Action: It is the first word in the signature, it can be drop/alert/pass... (for more details, visit <https://suricata.readthedocs.io/en/latest/configuration/suricata-yaml.html#suricata-yaml-action-order>)

Protocol: This keyword tells Suricata which protocol it concerns. It can be one of the four basic protocols: tcp, udp, icmp and ip (ip stands for 'all' or 'any') or one of so-called application layer protocols, as: http, ftp, tls (this includes ssl), smb, dns, ssh, smtp and more.

Source and destination: It specifies the traffic source and destination. Normally, you would also make use of variables, such as \$HOME_NET and \$EXTERNAL_NET. The configuration file specifies the IP addresses these concerns, and these settings will be used in place of the variables in you rules (see <https://suricata.readthedocs.io/en/latest/configuration/suricata-yaml.html#suricata-yaml-rule-vars> for more information).

Ports (source and destination): The first emphasized part is the source, the second is the destination.

Rule options: The rest of the rule consists of options. These are enclosed by parenthesis and separated by semicolons. Some options have settings (such as msg), which are specified by the keyword of the option, followed by a colon, followed by the settings. Others have no settings, and are simply the keyword (such as nocase):

<keyword>: <settings>;

<keyword>;

Flow: The flow keyword can be used to match on direction of the flow, to/from client or to/from server. It can also match if the flow is established or not.

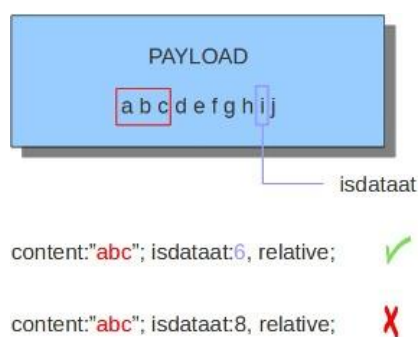
Content: The content keyword is very important in signatures. Between the quotation marks you can write on what you would like the signature to match. It is possible to use several contents in a signature.

Contents match on bytes. You can match on all

characters; But not all of the bytes are printable characters. For these bytes hexadecimal notations are used. Many programming languages use 0x00 as a notation, however the rule language uses |00| as a notation (|61| is a - |61 61| is aa - |41| is A - |21| is ! - |3A| is : and so on)

nocase: If you do not want to make a distinction between uppercase and lowercase characters, you can use nocase. The keyword nocase is a content modifier.

Isdataat: The purpose of the isdataat keyword is to look if there is still data at a specific part of the payload. The keyword starts with a number (the position) and then optional followed by 'relative' separated by a comma and the option rawbytes. You use the word 'relative' to know if there is still data at a specific part of the payload relative to the last match.



http_header: With the http_header content modifier, it is possible to match specifically and only on the HTTP header buffer. This contains all of the extracted headers in a single buffer.

Reference: The reference keywords direct to places where information about the signature and about the problem the signature tries to address, can be found. It has the following format: "reference: type, reference"

Classtype: The classtype keyword gives information about the classification of rules and alerts. It consists of a short name, a long name and a priority. For each classtype, the classification.config has a priority which will be used in the rule. (classtype definition: config classification: classtype, Alert, Priority)

sid (signature ID): The keyword sid gives every signature its own id. This id is stated with a number. The format of sid is: "sid:123;"

rev (revision): Represents the version of the signature. If a signature is modified, the number of rev will be incremented by the signature writers. The format of rev is: "rev:123;"

References

- [1] Pr. Slim Rekhis, Intrusion Detection: Network Intrusion Prevention Systems
Exploitation Use of Suricata IPS (Part 1)

